

Deep Reinforcement Learning and Applications

Modern algorithms and real-world challenges — Lecture 4/4

Etienne Chassaing

CentraleSupélec

1. Function Approximation
2. Deep Q-Learning
3. Policy Gradient Methods
4. Actor-Critic Methods
5. Modern Challenges
6. Case Studies

Learning Goals — Course 4

By the end of this lecture, you should understand:

1. **Function approximation** — why tables break down, and how neural networks fit into the bigger picture of parameterized functions
2. **Deep Q-Networks (DQN)** — extend Q-learning to continuous/large state spaces using neural networks
3. **Policy Gradient & Actor-Critic** — learn a policy directly by gradient ascent, and combine it with a learned critic
4. **Modern challenges** — alignment, exploration in high-dimensional spaces, simulation-to-reality gap
5. **Case studies** — application to optimal control problems (heat pipe, robotics, etc.)
6. **Practical lessons** — what works in practice, common pitfalls, when to use each algorithm

Context: Lectures 1–3 covered tabular methods. Here, we scale RL to real-world problems using neural networks and explore why deployment is hard.

Goals of today's session

Complete this slide with session-specific learning objectives

Lectures 1–3: Tabular RL

- Finite state/action spaces
- Tables to store V or Q
- Convergence proofs available
- Works for small problems

Challenge: Neural networks break many nice properties of tabular RL (convergence, stability). We need tricks (experience replay, target networks, etc.).

Lecture 4: Deep RL

- Continuous/very large state spaces
- Neural networks to approximate V or Q
- Heuristics work but no guarantees
- Real-world applications

1. Function Approximation
2. Deep Q-Learning
3. Policy Gradient Methods
4. Actor-Critic Methods
5. Modern Challenges
6. Case Studies

Why We Need Function Approximation

Tabular RL (Lectures 1–3): store one number per state (or state-action pair) in a table: $V(s)$, $Q(s, a)$, $\pi(a|s)$.

Breaks down when:

- $|S|$ is huge (Go: $\sim 10^{170}$) or infinite (continuous state, e.g. robot joint angles)
- States are never revisited exactly $\Rightarrow$ no generalization between similar states
- We want the agent to **generalize**: “I’ve never seen this exact state, but it looks like one I’ve seen”

Solution: Replace the table with a **parameterized function**:

$$\hat{v}(s; \mathbf{w}) \approx v^\pi(s), \quad \hat{q}(s, a; \mathbf{w}) \approx q^\pi(s, a), \quad \pi(a|s; \theta)$$

where $\mathbf{w}$ (or θ) is a **small** vector of parameters, $|\mathbf{w}| \ll |S|$.

Key shift: Learning becomes **optimizing parameters** (e.g. gradient descent) rather than filling table entries.

Parameterizing Functions: NN Is One Choice Among Others

Important: the RL algorithms (TD, Q-learning, policy gradient) don't care *how* $\hat{v}$, $\hat{q}$, or π are parameterized, only that we can compute a gradient w.r.t. the parameters.

Classic (pre-deep-learning) approach:

- Hand-craft a feature vector $x(s) \in \mathbb{R}^d$
- **Linear** function approximation:
$$\hat{v}(s; \mathbf{w}) = \mathbf{w}^\top x(s)$$
- Features: tile coding, radial basis functions, polynomial/Fourier basis, expert-designed indicators
- Used in Samuel's checkers player (1959), classic TD-Gammon features, early control systems

Neural network approach:

- $\hat{v}(s; \mathbf{w})$ = output of a network with weights $\mathbf{w}$
- Hidden layers **learn the features** $x(s)$ automatically, instead of hand-crafting them
- Same gradient-based update rules apply, just backprop through more layers
- Enables scaling to raw, high-dimensional inputs (pixels, joint sensors)

Takeaway: “Deep” RL = RL + neural nets as the function approximator. The RL ideas (Bellman

1. Function Approximation
2. Deep Q-Learning
3. Policy Gradient Methods
4. Actor-Critic Methods
5. Modern Challenges
6. Case Studies

Deep Q-Networks (DQN)

Main idea: Replace lookup table $Q(s, a)$ with a neural network $Q(s, a; \theta)$.

Naive approach fails:

- TD updates are highly correlated (sequential states)
- Bootstrapping on unstable target network diverges
- Learning is inefficient

DQN tricks:

1. **Experience Replay:** Store (s, a, r, s') in buffer, sample random minibatches to break correlation
2. **Target Network:** Separate network $Q(s', a'; \theta^-)$ for computing targets, update slowly
3. **Clipped Rewards:** Normalize rewards for stability

Update rule:

$$Q(s, a; \theta) \leftarrow Q(s, a; \theta) + \alpha[r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)]$$

Experience Replay & Target Networks

Why these tricks matter:

Experience Replay:

- Breaks temporal correlation
- Allows reuse of old data
- More sample-efficient

Implementation:

- Store last N transitions
- Sample B random batches
- Update network on batch

Target Network:

- Q network updates online
- Q^- network updates slowly (every C steps)
- Stabilizes target

Why works:

- Without it: target shifts as Q learns
- Result: oscillation or divergence
- Lagged target = more stable training

1. Function Approximation
2. Deep Q-Learning
- 3. Policy Gradient Methods**
4. Actor-Critic Methods
5. Modern Challenges
6. Case Studies

Why Learn a Policy Directly?

So far (DQN): learn $Q(s, a; \mathbf{w})$, then act greedily: $\pi(s) = \arg \max_a Q(s, a; \mathbf{w})$.

Limitations of value-based methods:

- $\arg \max_a$ is expensive or ill-defined for **continuous action spaces** (e.g. torque, throttle)
- Optimal policies can be **stochastic** (e.g. Rock-Paper-Scissors), greedy-on-Q can't represent that
- Small changes in Q can flip the $\arg \max$ discontinuously $\Rightarrow$ unstable policy behaviour

New idea: parameterize the **policy itself** directly, and adjust its parameters by gradient ascent on performance:

$$\pi(a|s; \theta), \quad J(\theta) = \mathbb{E}_{\pi_\theta} [G_0], \quad \theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Same idea as function approximation: θ can parameterize π via a linear model, a neural network, or anything differentiable.

The Policy Gradient Theorem

Challenge: $J(\theta)$ depends on which states get visited, which depends on π_θ itself, so it seems to require differentiating the state distribution too.

Policy Gradient Theorem (Sutton & Barto, 13.2): the gradient can be written **without** differentiating the state distribution:

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{\pi_\theta} [q^{\pi_\theta}(s, a) \nabla_\theta \log \pi(a|s; \theta)]$$

Reading it:

- $\nabla_\theta \log \pi(a|s; \theta)$: direction in parameter space that makes action a **more likely** in state s
- $q^{\pi_\theta}(s, a)$: weight, push harder in that direction if the action turned out **good**
- Net effect: **good actions become more likely, bad actions less likely**

Monte Carlo estimate (REINFORCE): replace $q^{\pi_\theta}(s, a)$ with the observed return G_t from that state onward.

REINFORCE (Monte Carlo Policy Gradient)

Input: Differentiable policy $\pi(a|s; \theta)$, step size $\alpha > 0$

Initialize θ arbitrarily

repeat

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ following $\pi(\cdot|\cdot; \theta)$

for each step $t = 0, \dots, T - 1$:

$$G_t \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$$

▷ return from step t

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi(A_t | S_t; \theta)$$

until convergence

Pure Monte Carlo: waits for the full episode, uses the actual return G_t ; unbiased but high variance.

Reducing Variance: Baselines

Problem with REINFORCE: G_t is noisy (full-episode Monte Carlo return) $\Rightarrow$ slow, unstable learning.

Fix: subtract a **baseline** $b(s)$ that doesn't depend on the action. This doesn't bias the gradient (its expectation is zero) but can reduce variance a lot:

$$\nabla_{\theta} J(\theta) \propto \mathbb{E}_{\pi_{\theta}} \left[\left(q^{\pi_{\theta}}(s, a) - b(s) \right) \nabla_{\theta} \log \pi(a|s; \theta) \right]$$

Natural choice: $b(s) = \hat{v}(s; \mathbf{w})$, an estimated **state value**, learned alongside the policy.

- The quantity $q^{\pi}(s, a) - v^{\pi}(s)$ is called the **advantage** $A^{\pi}(s, a)$: “how much better than average was this action?”
- Learning $\hat{v}(s; \mathbf{w})$ requires a second parameterized function $\Rightarrow$ we now have **two** learners working together

This is exactly the setup of **Actor-Critic** methods.

1. Function Approximation
2. Deep Q-Learning
3. Policy Gradient Methods
- 4. Actor-Critic Methods**
5. Modern Challenges
6. Case Studies

Idea: Learn two parameterized functions simultaneously (either can be linear, NN, or any other differentiable model):

- **Actor** $\pi(a|s; \theta)$: the policy (what action to take?), updated via policy gradient
- **Critic** $\hat{v}(s; \mathbf{w})$: the value estimate (how good is this state?), updated via TD, exactly as in Lecture 3

Why “critic” instead of the raw return G_t ?

- Critic gives a **TD error** $\delta = r + \gamma \hat{v}(s'; \mathbf{w}) - \hat{v}(s; \mathbf{w})$ as a low-variance stand-in for the advantage $A^\pi(s, a)$
- Available **every step** (bootstrapping), not just at episode end $\Rightarrow$ can learn **online**, without waiting for a full episode

Trade-off vs. REINFORCE: lower variance (critic reduces noise), at the cost of some bias (bootstrapping on an imperfect critic).

One-Step Actor-Critic

Input: Differentiable policy $\pi(a|s; \theta)$ and value function $\hat{v}(s; \mathbf{w})$, step sizes $\alpha_\theta, \alpha_w > 0$

Initialize $\theta, \mathbf{w}$ arbitrarily

repeat

Initialize S (first state of episode)

repeat

$A \sim \pi(\cdot|S; \theta)$; take A , observe R, S'

$\delta \leftarrow R + \gamma \hat{v}(S'; \mathbf{w}) - \hat{v}(S; \mathbf{w})$

$\mathbf{w} \leftarrow \mathbf{w} + \alpha_w \delta \nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w})$

$\theta \leftarrow \theta + \alpha_\theta \delta \nabla_{\theta} \log \pi(A|S; \theta)$

$S \leftarrow S'$

until S is terminal

until convergence

▷ for each step of episode

▷ TD error $\approx$ advantage

▷ critic update

▷ actor update

Both updates fire at **every step** using the same TD error δ : fully online, no need to wait for episode end.

1. Function Approximation
2. Deep Q-Learning
3. Policy Gradient Methods
4. Actor-Critic Methods
- 5. Modern Challenges**
6. Case Studies

The Alignment Problem

Definition: Agent optimizes reward function but deviates from designer intent.

Classic examples:

- Atari: Agent learns to exploit glitches instead of playing well
- Grasping robot: Moves camera instead of moving arm
- Dialogue agent: Manipulates rewards through hacking

Root cause: Reward function captures only measurable proxy, not true objective.

Partial solutions:

- Better reward design (harder than it sounds)
- Constrained optimization (stay close to safe region)
- Human feedback (RLHF)
- Interpretability (understand what agent learned)

Open problem: No silver bullet for alignment in open-ended environments.

Simulation-to-Reality Gap

Problem: Agent trained in simulation performs poorly on real robot.

Sources of gap:

- Physics errors in simulator (friction, inertia, collisions)
- Sensor noise and delays not modeled
- Actuator characteristics (dead zones, limits)
- Lighting, textures, camera calibration

Solutions:

1. **Domain Randomization:** Vary simulation parameters during training
 - ▶ Force agent to learn robust policy across many conditions
 - ▶ Works surprisingly well in practice
2. **Sim-to-Real Transfer:** Fine-tune on real robot with RL
3. **Model Adaptation:** Learn correction function from real data

Reality: Most production systems combine simulation + real-world data.

1. Function Approximation
2. Deep Q-Learning
3. Policy Gradient Methods
4. Actor-Critic Methods
5. Modern Challenges
- 6. Case Studies**

Case Study 1: Heat Pipe Control

Problem: Control temperature in a heat pipe system (complex nonlinear dynamics).

Why RL?

- No accurate physics model
- Tuning PID gains is tedious
- System has constraints (power, safety bounds)
- Data-driven approach preferred

MDP Formulation:

- State: temperature + rate of change
- Action: heater power (0–100%)
- Reward: $-(|T - T_{\text{target}}| + 0.1 \times \text{power}^2)$

Results:

- Q-Learning: Fast learning, may overshoot
- SARSA: More stable, conservative
- Actor-Critic: Smooth control, best long-term

Key lesson: Different algorithms suit different objectives. Safety-critical systems favor conservative methods.

Ref: https://etiennechassaing.com/book/case-studies/heat_pipe_control/

Key Takeaways

- **Function approximation:** tables $\rightarrow$ parameterized functions; neural nets are one parametrization among others (linear features, tile coding, ...)
- **Deep RL:** Scales to continuous/large spaces but needs tricks (replay, target nets)
- **Policy Gradient:** learn $\pi(a|s; \theta)$ directly via $\nabla_{\theta} J(\theta) \propto \mathbb{E}[q^{\pi}(s, a) \nabla_{\theta} \log \pi(a|s; \theta)]$; REINFORCE is the Monte Carlo instance
- **Actor-Critic:** Actor (policy) + Critic (value) trained together; TD error as low-variance advantage estimate, fully online
- **Alignment:** Reward design is hard; misspecified rewards cause agent to exploit
- **Sim-to-Real:** Domain randomization helps; real-world fine-tuning often necessary
- **Practical:** Choose algorithm based on problem structure (safety, sample efficiency, interpretability)

Thank you!

Questions?

`etienne.chassaing@centralesupelec.fr`



Course materials and references:

`github.com/cetiennec/cours-pdf`



Next: Independent projects using RL!

About this course: I do use LLMs/AI to create this course, with the following uses:

- Format $\text{\LaTeX}$ slides and exercises
- Find relevant sources
- Format equations and notations

However, plan and ideas are human-based, and everything content has been carefully reviewed. If you find some errors, or inconsistencies, please report them to me.

What about your own LLM usage?

Of course you also have the right to use LLMs, but don't forget you're in a learning step. Learning process goes through effort and the newer the subject, the more one needs empty-sheet recall.

- * See: Desirable difficulty
- * Chi & Wylie (2014), ICAP hypothesis